

PRODUCT MANAGEMENT API (IN-MEMORY) – DOCUMENTATION

THEORY: WHAT IS THIS PROJECT?

This project implements a **Product Management REST API** using **Django**.
The API allows clients to:

- Create a product
- Fetch all products
- Fetch a single product
- Update a product
- Delete a product

Important:

All operations are **in-memory**, meaning **no database** is used.
Data exists **only while the server is running**.

THEORY: WHAT IS AN IN-MEMORY API?

An **in-memory API** stores data in program memory (RAM) instead of a database.

Characteristics:

- Data stored in Python variables (lists, dictionaries)
- Faster for learning & testing
- Data is lost when server stops
- Useful for:

- Assignments
- Interviews
- Prototyping

In this project:

```
products = []
```

acts as our **temporary database**.

3 THEORY: WHY DJANGO WITHOUT DATABASE?

Normally Django uses ORM + database, but here:

- The requirement explicitly says **in-memory operations**
 - We use Django **only for routing, request handling, and responses**
 - The Product class is a **plain Python class**, not a Django model
-

4 PROJECT STRUCTURE

```
product/  
|— data.py      # in-memory storage  
|— models.py    # Product structure  
|— views.py     # API logic  
|— urls.py      # API routes
```

FILE 1: data.py (IN-MEMORY STORAGE)

```
# in memory storage
```

➡ Comment explaining that data is stored in RAM.

```
# all the data related to the product will be stored here on calling
```

➡ Clarifies that all product data will be kept here.

```
# also all products will be stored only while server is running
```

➡ Explains limitation: data disappears when server stops.

```
products = []
```

➡ Creates an empty Python list

➡ Acts like a **temporary database table**

➡ Each element will be a product dictionary

Example:

```
{  
    "id": 1,  
    "name": "Phone",  
    "price": 20000  
}
```

FILE 2: **models.py** (PRODUCT STRUCTURE)

```
from django.db import models
```

➡ Imported by default

➡ ❌ Not used because no database is involved

```
class Product:
```

- ➡ Defines a **plain Python class**
- ➡ Used to structure product data

```
def __init__(self, id, name, desc, category, price, brand, quantity):
```

- ➡ Constructor method
- ➡ Runs automatically when a Product object is created

```
self.id = id
```

- ➡ Stores unique product ID

```
self.name = name
```

- ➡ Stores product name

```
self.desc = desc
```

- ➡ Stores product description

```
self.category = category
```

- ➡ Stores product category

```
self.price = price
```

- ➡ Stores product price

```
self.brand = brand
```

➡ Stores brand name

```
self.quantity = quantity
```

➡ Stores available stock in warehouse

📌 Why not Django Model?

Because data is not saved in DB; we only need a data structure.

FILE 3: `urls.py` (API ROUTING)

```
from django.urls import path
```

➡ Imports Django URL routing utility

```
from . import views
```

➡ Imports all view functions

```
urlpatterns = [
```

➡ Django reads this list to map URLs to functions

```
path("products/", views.create_product),
```

➡ URL: `/products/`

➡ Handles:

- GET → list products
- POST → create product

```
path("products/<int:id>/", views.get_product),
```

➡ `<int:id>` captures product ID from URL

➡ Example: `/products/2/`

```
path("products/<int:id>/update/", views.update_product),
```

➡ URL for updating a product

➡ Uses PUT request

```
path("products/<int:id>/delete/", views.delete_product),
```

➡ URL for deleting a product

➡ Uses DELETE request

```
]
```

➡ End of URL patterns



FILE 4: `views.py` (API LOGIC)

◆ Imports

```
import json
```

➡ Converts JSON request body → Python dictionary

```
from django.http import JsonResponse
```

➡ Sends JSON responses instead of HTML

```
from django.views.decorators.csrf import csrf_exempt
```

- ➡ Disables CSRF protection
- ➡ Needed for Postman testing

```
from .models import Product
```

- ➡ Imports Product class

```
from .data import products
```

- ➡ Imports in-memory product list

◆ **create_product()**

```
@csrf_exempt  
def create_product(request):
```

- ➡ CSRF disabled
- ➡ Handles GET & POST

GET REQUEST

```
if request.method == "GET":
```

- ➡ Checks HTTP method

```
return JsonResponse({"products": products})
```

- ➡ Returns all stored products
-

POST REQUEST

```
if request.method == "POST":
```

➡ Runs for product creation

```
data = json.loads(request.body)
```

➡ Converts JSON body into dictionary

```
required_fields = ["name", "price", "category", "brand", "quantity"]
```

➡ Mandatory fields list

```
for field in required_fields:
```

➡ Loops through required fields

```
if field not in data:
```

➡ Validation check

```
return JsonResponse({"error": f"{field} is required"}, status=400)
```

➡ Returns error if missing field

```
if data["price"] <= 0:
```

➡ Business validation

```
return JsonResponse({"error": "Price must be positive"}, status=400)
```


➡ Error response

```
product = Product(
```

➡ Creates Product object

```
id=len(products) + 1,
```

➡ Auto-increments ID

```
name=data["name"],  
description=data.get("description", ""),
```

➡ Optional description

```
category=data["category"],  
price=data["price"],  
brand=data["brand"],  
quantity=data["quantity"]
```

➡ Assign remaining attributes

```
products.append(product.__dict__)
```

➡ Converts object → dictionary

➡ Stores in memory list

```
return JsonResponse({"message": "Product created", "product":  
product.__dict__}, status=201)
```

➡ Success response

```
return JsonResponse({"error": "Method not allowed"}, status=405)
```

➡ Handles invalid HTTP methods

◆ **get_product()**

```
def get_product(request, id):
```

➡ Receives product ID from URL

```
if request.method != "GET":
```

➡ Allows only GET

```
for product in products:
```

➡ Loops through product list

```
if product["id"] == id:
```

➡ Matches ID

```
return JsonResponse(product)
```

➡ Returns product

```
return JsonResponse({"error": "Product not found"}, status=404)
```

➡ If not found

◆ **update_product()**

```
if request.method != "PUT":
```

➡ Only PUT allowed

```
data = json.loads(request.body)
```

➡ Updated data from client

```
product.update(data)
```

➡ Updates dictionary fields

```
return JsonResponse({"message": "Product Updated", "product":  
product})
```

➡ Confirmation response

◆ **delete_product()**

```
if request.method != "DELETE":
```

➡ Only DELETE allowed

```
global products
```

➡ Allows modifying global list

```
products = [p for p in products if p["id"] != id]
```

➡ Removes product by ID

```
return JsonResponse({"message": "Product deleted"})
```

➡ Confirms deletion

ASSIGNMENT CHECKLIST

Requirement	Status
Product model	
In-memory operations	
CRUD APIs	
Validations	
HTTP client testable	